NEXT ENGINEER · _SHEERAT_

# Application of Soft Computing

BCS056

One-day revision notes: definitions, solved examples, code and important questions

**Program:** B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 5 | **Credits:** 3

**L-T-P:** 3-0-0 | **Scheme:** AKTU B.Tech 3rd year, Departmental Elective, 2024-25 onwards


**UNIT 1** Neural Network Fundamentals

**UNIT 2** Backpropagation Networks

**UNIT 3** Fuzzy Logic Basics

**UNIT 4** Fuzzy Membership, Rules and Controllers

**UNIT 5** Genetic Algorithms


Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: models, learning rules and comparison tables.
3. **Solved examples**: the numericals that appear (McCulloch-Pitts, Hebb, perceptron, backprop step, fuzzy operations, max-min composition, defuzzification, GA cycle).
4. **Formulas to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

## How to use these notes the day before the paper

- **Morning (3 hours):** Units 1 and 2. Biological vs artificial neuron, activation functions, architectures, McCulloch-Pitts and perceptron, Hebb rule numerical, associative memory; backpropagation steps and factors affecting training.
- **Afternoon (3 hours):** Units 3 and 4. Crisp vs fuzzy sets, operations and properties, fuzzy relations and max-min composition, membership functions, fuzzy inference (Mamdani), defuzzification methods, a fuzzy controller design.
- **Evening (2 hours):** Unit 5. GA terminology, encoding, selection, crossover, mutation, the generational cycle and applications. Then the "Last-hour checklist".
- **In the exam:** for fuzzy numericals, write every element with its membership value; for NN numericals, show the weight table after each input.

**AKTU paper pattern (theory, 70 marks, 3 hours):** Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

# UNIT 1 Neural Network Fundamentals

## ✍️ 2-mark definitions

- **Soft computing:** a collection of methods (neural networks, fuzzy logic, genetic algorithms, probabilistic reasoning) that tolerate imprecision, uncertainty and partial truth to give low-cost, robust solutions. Hard computing needs exact models and precise data.
- **Biological neuron:** a nerve cell with dendrites (receive signals), soma (cell body that sums signals), axon (sends output) and synapses (junctions that weight the signal).
- **Artificial neuron:** a model that computes a weighted sum of inputs plus bias, net = $\Sigma\, w_i x_i + b$, and applies an activation function y = f(net).
- **Activation function:** a function that converts the net input into the neuron's output: identity, binary step, bipolar step, sigmoid (binary and bipolar), ramp.
- **McCulloch-Pitts (M-P) neuron:** the first neuron model (1943) with binary inputs, fixed excitatory/inhibitory weights and a threshold; it fires if the net input reaches the threshold.
- **Perceptron:** a single-layer network with adjustable weights trained by the perceptron learning rule; it solves only linearly separable problems.
- **Learning:** adjusting weights so the network produces the desired output: supervised, unsupervised, or reinforcement.
- **Hebbian learning:** "neurons that fire together, wire together": $\Delta w_i = x_i\, y$.
- **Associative memory:** a network that stores pattern pairs and recalls a stored pattern from a noisy or partial input. Auto-associative recalls the same pattern; hetero-associative recalls a different, paired pattern.
- **Hopfield network:** a recurrent, fully connected, auto-associative network with symmetric weights and no self-connections.
- **Linear separability:** two classes can be separated by a straight line (hyperplane); AND and OR are linearly separable, XOR is not.

## 💡 Core concepts

### Biological vs artificial neural network

| Biological | Artificial |
|---|---|
| Dendrites | inputs |
| Synapse | weights |
| Soma | summation (net input) |
| Axon | output |
| Firing threshold | activation function / threshold |

### Network architectures

| Architecture | Description | Example |
|---|---|---|
| Single-layer feed-forward | input layer directly to output layer | perceptron, Adaline |
| Multilayer feed-forward | one or more hidden layers, no feedback | MLP (backprop) |
| Recurrent (feedback) | outputs fed back as inputs | Hopfield, Elman |
| Competitive | output neurons compete; one wins | Kohonen SOM |

### Activation functions

- Binary step: f(x) = 1 if x ≥ θ, else 0.
- Bipolar step: f(x) = 1 if x ≥ θ, else −1.
- Binary sigmoid: f(x) = 1/(1 + e^(−λx)), derivative λf(1 − f).
- Bipolar sigmoid: f(x) = (1 − e^(−λx))/(1 + e^(−λx)), derivative (λ/2)(1 + f)(1 − f).

### Learning rules

| Rule | Weight change | Type |
|---|---|---|
| Hebb | Δw = η x y | unsupervised (or with target) |
| Perceptron | Δw = η (t − y) x, only when y ≠ t | supervised |
| Delta (Widrow-Hoff, LMS) | Δw = η (t − y_in) x | supervised |
| Competitive (winner takes all) | Δw = η (x − w) for winner | unsupervised |

**Applications of NNs:** pattern and character recognition, speech recognition, forecasting, medical diagnosis, control, data compression.

## 🧮 Solved examples

**Example 1 (M-P neuron for AND).** Weights w1 = w2 = 1, threshold θ = 2. Inputs (1, 1) give net 2 ≥ 2, output 1; (1, 0), (0, 1) give 1 < 2, output 0; (0, 0) gives 0. This is AND. With θ = 1 the same neuron gives OR.

**Example 2 (Hebb net for AND, bipolar).** Start with w1 = w2 = b = 0; Δw = x·t.

| x1 | x2 | t | Δw1 | Δw2 | Δb | w1 | w2 | b |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | −1 | −1 | −1 | 1 | −1 | 0 | 2 | 0 |
| −1 | 1 | −1 | 1 | −1 | −1 | 1 | 1 | −1 |
| −1 | −1 | −1 | 1 | 1 | −1 | 2 | 2 | −2 |

Final weights **w1 = 2, w2 = 2, b = −2**. Decision line: 2×1 + 2×2 − 2 = 0, i.e. x1 + x2 = 1.

**Example 3 (perceptron update).** η = 1, θ = 0, w = (0, 0), b = 0, bipolar step. Input (1, 1), target 1: net 0, output 1 if net ≥ 0, so correct with this convention; for input (1, −1), target −1: net 0 → output 1 ≠ −1, so update w = (0, 0) + (−1)(1, −1) = (−1, 1), b = −1.

**Example 4 (XOR).** XOR outputs 1 for (0, 1) and (1, 0) and 0 for (0, 0) and (1, 1). No single line separates these, so a single perceptron cannot learn it; a two-layer network (e.g. hidden units computing OR and NAND, output AND) can.

## Formulas to remember

- net = $\Sigma\, w_i x_i + b$; y = f(net).
- Hebb: w(new) = w(old) + x t.
- Perceptron: w(new) = w(old) + $\eta$ t x (when output ≠ t).
- Hopfield energy: $E = -\tfrac{1}{2} \Sigma_i \Sigma_j w_{ij} s_i s_j$ (decreases as the network updates).

## ⭐ Important questions

1. Define soft computing and compare it with hard computing; list its constituents.
2. Compare biological and artificial neurons; explain the model of an artificial neuron with activation functions.
3. Explain NN architectures (single-layer, multilayer, recurrent).
4. Explain the McCulloch-Pitts neuron and the perceptron with learning rule; design networks for AND/OR.
5. Explain Hebbian learning and associative memories (auto, hetero, Hopfield).

NEXT ENGINEER _sheerat_

# UNIT 2 Backpropagation Networks

## ✍️ 2-mark definitions

- **Multilayer perceptron (MLP):** a feed-forward network with input, hidden and output layers using differentiable (sigmoid) activations.
- **Backpropagation (generalized delta rule):** a supervised algorithm that propagates the output error backward to compute weight changes for all layers by gradient descent.
- **Learning rate (η, learning coefficient):** controls the step size of weight updates; too large causes oscillation, too small makes learning slow.
- **Momentum (α):** adds a fraction of the previous weight change to the current one to speed up learning and smooth oscillation: $\Delta w(t) = \eta\, \delta\, x + \alpha\, \Delta w(t-1)$.
- **Epoch:** one pass of all training patterns through the network.
- **Local minimum:** a point where the error is lower than nearby points but not the lowest overall; the network may get stuck there.
- **Generalization:** the ability of a trained network to give correct outputs for inputs it has not seen.
- **Overtraining (overfitting):** training too long, so the network memorizes training data and generalizes poorly.

## 💡 Core concepts

**Architecture of a backpropagation network:** input layer (no processing), one or more hidden layers, output layer; every unit in one layer connects to every unit in the next; biases on hidden and output units; binary or bipolar sigmoid activations.

**Backpropagation algorithm (binary sigmoid)**

1. Initialize weights and biases to small random values.
2. **Feed-forward:** each hidden unit computes $z_in = v0 + \Sigma\, x_i v_{ij}$ and $z = f(z_in)$; each output unit computes $y_in = w0 + \Sigma\, z_j w_{jk}$ and $y = f(y_in)$.
3. **Error at output:** $\delta k = (tk - yk)\, f'(y_in) = (tk - yk)\, yk(1 - yk)$.
4. **Error at hidden units:** $\delta j = (\Sigma\, \delta k\, w_{jk})\, z_j(1 - z_j)$.
5. **Update weights:** $w_{jk} \leftarrow w_{jk} + \eta\, \delta k\, z_j$; $v_{ij} \leftarrow v_{ij} + \eta\, \delta j\, x_i$ (and biases with input 1).
6. Repeat for all patterns until the total error is small or the epoch limit is reached.

**Factors affecting backpropagation training**

- Initial weights: small random values (e.g. −0.5 to 0.5); Nguyen-Widrow initialization speeds learning.
- Learning rate: typical 0.1 to 0.9; adaptive rates help.
- Momentum: typical 0.5 to 0.9.
- Number of hidden layers and units: too few cannot learn; too many overfit and slow training.
- Number and quality of training patterns, and normalization of inputs.
- Activation function choice and stopping criterion.

**Merits and demerits:** can learn any non-linear mapping (universal approximator) and is widely used; but training is slow, can get stuck in local minima, needs many examples, and network size is chosen by trial.

**Applications:** character and speech recognition, image compression, forecasting, medical diagnosis, robot control.

## 🧮 Solved examples

**Example 1 (one backprop step).** One input x = 1, one hidden unit with weight v = 0.5 (no bias), one output unit with weight w = 0.4 (no bias), target t = 1, η = 0.5, binary sigmoid.

- Hidden: z_in = 0.5, z = 1/(1 + e^(−0.5)) ≈ 0.622.
- Output: y_in = 0.4 × 0.622 ≈ 0.249, y ≈ 0.562.
- δ_out = (1 − 0.562) × 0.562 × 0.438 ≈ 0.108.
- δ_hidden = 0.108 × 0.4 × 0.622 × 0.378 ≈ 0.0102.
- New w = 0.4 + 0.5 × 0.108 × 0.622 ≈ **0.434**.
- New v = 0.5 + 0.5 × 0.0102 × 1 ≈ **0.505**.

**Example 2 (momentum).** Previous Δw = 0.02, current η δ x = 0.05, α = 0.9: Δw = 0.05 + 0.9 × 0.02 = **0.068**.

## Formulas to remember

- $f(x) = 1/(1 + e^{(-x)})$, $f'(x) = f(x)(1 - f(x))$.
- $\delta k = (tk - yk)\, yk(1 - yk)$; $\delta j = z_j(1 - z_j) \Sigma\, \delta k\, w_{jk}$.
- $\Delta w_{jk} = \eta\, \delta k\, z_j + \alpha\, \Delta w_{jk}(\text{previous})$.
- Total error $E = \tfrac{1}{2} \Sigma\, (tk - yk)^2$.

## ⭐ Important questions

1. Explain the architecture of a backpropagation network.
2. Explain the backpropagation learning algorithm step by step with its derivation of the delta rule.
3. Explain the effect of learning coefficient and momentum; perform one training step on given data.
4. Discuss the factors that affect backpropagation training.
5. Discuss merits, demerits and applications of backpropagation networks.

NEXT ENGINEER _sheerat_

# UNIT 3 Fuzzy Logic Basics

## ✍️ 2-mark definitions

- **Crisp set:** a set in which an element either belongs (membership 1) or does not (membership 0).
- **Fuzzy set:** a set in which each element has a membership degree μ(x) in [0, 1], written A = {μ(x1)/x1 + μ(x2)/x2 + ...}.
- **Universe of discourse:** the full range of values an element may take.
- **Support:** the elements with μ(x) > 0. **Core:** elements with μ(x) = 1.
- **Height:** the largest membership value; a fuzzy set is normal if its height is 1.
- **α-cut (λ-cut):** the crisp set of elements with μ(x) ≥ α.
- **Fuzzy relation:** a fuzzy set on the Cartesian product X × Y, giving the degree of relationship between pairs, written as a matrix.
- **Composition of relations:** combining R on X × Y and S on Y × Z into T on X × Z, usually by max-min composition.
- **Fuzzy logic:** a form of many-valued logic in which truth values range between 0 and 1.

## 💡 Core concepts

### Crisp vs fuzzy sets

| Point | Crisp | Fuzzy |
|---|---|---|
| Membership | 0 or 1 | any value in [0, 1] |
| Boundary | sharp | gradual |
| Law of excluded middle | holds (A ∪ A′ = X) | does not hold |
| Law of contradiction | holds (A ∩ A′ = ∅) | does not hold |
| Example | "students above 18 years" | "tall students" |

### Fuzzy set operations

- Union: μ(A ∪ B)(x) = max(μA(x), μB(x)).
- Intersection: μ(A ∩ B)(x) = min(μA(x), μB(x)).
- Complement: μ(A′)(x) = 1 − μA(x).
- Algebraic sum: μA + μB − μA μB. Algebraic product: μA μB.
- Bounded sum: min(1, μA + μB). Bounded difference: max(0, μA − μB).

**Properties that hold for fuzzy sets:** commutativity, associativity, distributivity, idempotency (A ∪ A = A), identity (A ∪ ∅ = A, A ∩ X = A), involution ((A′)′ = A), transitivity, and De Morgan's laws. Excluded middle and contradiction do **not** hold.

### Fuzzy relations

- Max-min composition: μT(x, z) = max over y of min(μR(x, y), μS(y, z)).

- Max-product composition: μT(x, z) = max over y of μR(x, y) · μS(y, z).
- Properties of relations: reflexive, symmetric, transitive. A tolerance relation is reflexive and symmetric; an equivalence relation is also transitive.

## 🧮 Solved examples

**Example 1 (operations).** A = {0.2/x1, 0.5/x2, 0.8/x3}, B = {0.6/x1, 0.3/x2, 0.9/x3}.

- A ∪ B = {0.6/x1, 0.5/x2, 0.9/x3}.
- A ∩ B = {0.2/x1, 0.3/x2, 0.8/x3}.
- A′ = {0.8/x1, 0.5/x2, 0.2/x3}.
- A ∪ A′ = {0.8, 0.5, 0.8} ≠ X, showing excluded middle fails.
- Algebraic product A·B = {0.12, 0.15, 0.72}.
- α-cut A(0.5) = {x2, x3}.

**Example 2 (max-min composition).** R = [0.6 0.3; 0.2 0.9] on X × Y; S = [1 0.5 0.3; 0.8 0.4 0.7] on Y × Z.

- T11 = max(min(0.6, 1), min(0.3, 0.8)) = max(0.6, 0.3) = 0.6.
- T12 = max(min(0.6, 0.5), min(0.3, 0.4)) = 0.5.
- T13 = max(min(0.6, 0.3), min(0.3, 0.7)) = 0.3.
- T21 = max(min(0.2, 1), min(0.9, 0.8)) = 0.8.
- T22 = max(min(0.2, 0.5), min(0.9, 0.4)) = 0.4.
- T23 = max(min(0.2, 0.3), min(0.9, 0.7)) = 0.7.
- **T = [0.6 0.5 0.3; 0.8 0.4 0.7]**.

**Example 3 (Cartesian product relation).** A = {0.3/a, 0.8/b}, B = {0.5/p, 1/q}. R = A × B with μ = min: R = [0.3 0.3; 0.5 0.8].

## Formulas to remember

- max for union, min for intersection, 1 − μ for complement.
- Max-min: T = R ∘ S, μT = max min.
- Cardinality of a fuzzy set: |A| = Σ μA(x).

## ⭐ Important questions

1. Compare crisp and fuzzy sets; explain membership and the universe of discourse.
2. Explain fuzzy set operations and properties with examples; show which crisp laws fail.
3. Explain fuzzy relations and their properties (reflexive, symmetric, transitive).
4. Find the max-min and max-product composition of given fuzzy relations.
5. Explain crisp relations vs fuzzy relations with Cartesian products.

NEXT ENGINEER _sheerat_

# UNIT 4 Fuzzy Membership, Rules and Controllers

## ✍️ 2-mark definitions

- **Membership function (MF):** a curve that maps each input value to a membership degree between 0 and 1.
- **Fuzzification:** converting a crisp input into membership degrees of fuzzy sets (e.g. temperature 30 °C is "warm" with 0.6 and "hot" with 0.2).
- **Linguistic variable:** a variable whose values are words, e.g. temperature = {cold, warm, hot}.
- **Fuzzy rule (if-then rule):** a statement like "IF temperature is hot AND humidity is high THEN fan speed is fast".
- **Fuzzy inference:** the process of mapping inputs to outputs using fuzzy rules (Mamdani or Sugeno).
- **Defuzzification:** converting the fuzzy output set into a single crisp value.
- **Fuzzy controller (FLC):** a control system using fuzzification, a rule base, an inference engine and defuzzification.
- **Hedge:** a modifier of a fuzzy set, e.g. "very" ($\mu^2$) and "somewhat" ($\sqrt{\mu}$).

## 💡 Core concepts

### Common membership functions

- Triangular (a, b, c): $\mu = \max(\min((x - a)/(b - a), (c - x)/(c - b)), 0)$.
- Trapezoidal (a, b, c, d): rises from a to b, flat 1 from b to c, falls from c to d.
- Gaussian: $\mu = \exp(-(x - c)^2/2\sigma^2)$.
- Sigmoidal and bell-shaped functions.

Features: core ($\mu = 1$), support ($\mu > 0$), boundaries ($0 < \mu < 1$).

**Methods of assigning membership values:** intuition, inference, rank ordering, angular fuzzy sets, neural networks, genetic algorithms and inductive reasoning.

### Mamdani fuzzy inference (steps)

1. Fuzzify the crisp inputs.
2. Apply fuzzy operators: AND = min, OR = max, to get each rule's firing strength.
3. Implication: clip (min) or scale each rule's output set by its strength.
4. Aggregate all rule outputs with max.
5. Defuzzify to get a crisp output.

Sugeno inference uses output functions ($z = ax + by + c$ or a constant) and a weighted average instead of defuzzifying a shape.

### Defuzzification methods

| Method | Formula / idea |
|---|---|
| Centroid (centre of gravity) | $z^* = \Sigma \mu(z)\, z / \Sigma \mu(z)$ (or $\int \mu\, z\, dz / \int \mu\, dz$) |

| Method | Formula / idea |
|---|---|
| Weighted average | $z^* = \Sigma\, \mu(\bar{z})\, \bar{z} / \Sigma\, \mu(\bar{z})$ using each set's centre $\bar{z}$ (symmetric sets) |
| Mean of maximum | average of the z values with the highest membership |
| Max membership (height) | z where μ is largest |
| Centre of sums, centre of largest area, first/last of maxima | variants |

**Fuzzy logic controller structure:** crisp inputs → fuzzification interface → knowledge base (database of MFs + rule base) → decision-making (inference) unit → defuzzification interface → crisp control output to the plant. Applications: washing machines, air conditioners, cameras (autofocus), train braking, cement kilns, traffic signals.

## 🧮 Solved examples

**Example 1 (triangular membership).** "Warm" = triangle (20, 30, 40) °C. At 25 °C: (25 − 20)/(30 − 20) = **0.5**. At 36 °C: (40 − 36)/(40 − 30) = **0.4**.

**Example 2 (rule evaluation).** Rule: IF temperature is warm (0.5) AND humidity is high (0.7) THEN fan is fast. Firing strength = min(0.5, 0.7) = **0.5**, so the "fast" output set is clipped at 0.5. With OR the strength would be max = 0.7.

**Example 3 (centroid defuzzification, discrete).** Output values z = 1, 2, 3, 4 with μ = 0.2, 0.6, 1.0, 0.4.

- $\Sigma\, \mu z = 0.2 + 1.2 + 3.0 + 1.6 = 6.0$.
- $\Sigma\, \mu = 2.2$.
- $z^* = 6.0/2.2 \approx$ **2.73**.
- Mean of maximum: only z = 3 has μ = 1, so $z^* = 3$.

**Example 4 (hedges).** μ(tall) = 0.8: "very tall" = 0.64; "somewhat tall" = $\sqrt{0.8} \approx 0.89$.

**Example 5 (washing machine controller, design outline).** Inputs: dirtiness {low, medium, high} and load {small, medium, large}. Output: wash time {short, medium, long}. Rules: IF dirt is high AND load is large THEN time is long; IF dirt is low AND load is small THEN time is short; etc. Fuzzify sensor values, evaluate rules with min, aggregate with max, and defuzzify with the centroid to get minutes.

## Formulas to remember

- Triangular MF as above; Gaussian $\exp(-(x - c)^2/2\sigma^2)$.
- AND = min, OR = max, NOT = 1 − μ.
- Centroid $z^* = \Sigma\, \mu\, z / \Sigma\, \mu$.

## ⭐ Important questions

1. What are membership functions? Explain their features and types (triangular, trapezoidal, Gaussian).
2. Explain fuzzification and methods of assigning membership values.

3. Explain fuzzy if-then rules and the Mamdani inference process; compare Mamdani and Sugeno.
4. Explain defuzzification methods; compute the crisp output for given data.
5. Explain the structure of a fuzzy logic controller with an application (washing machine, air conditioner).

NEXT ENGINEER
_sheerat_

# UNIT 5 Genetic Algorithms

## ✍️ 2-mark definitions

- **Genetic algorithm (GA):** a search and optimization method based on natural selection and genetics, introduced by John Holland (1975).
- **Population:** a set of candidate solutions (chromosomes) in one generation.
- **Chromosome, gene, allele:** a chromosome encodes a solution; a gene is one position; an allele is the value at that position.
- **Genotype and phenotype:** the encoded string and the actual solution it represents.
- **Fitness function:** measures how good a chromosome is; GA tries to maximize it.
- **Encoding:** the way solutions are represented: binary, octal, hexadecimal, permutation (value order), value encoding, tree encoding.
- **Selection:** choosing parents with a bias toward fitter chromosomes.
- **Crossover (recombination):** creating offspring by exchanging genes between two parents.
- **Mutation:** randomly changing genes to maintain diversity and avoid premature convergence.
- **Elitism:** copying the best chromosomes unchanged into the next generation.
- **Schema (Holland):** a template such as 1*0* describing a set of strings; short, low-order, above-average schemata grow over generations (schema theorem).

## 💡 Core concepts

### Generational cycle

1. Encode and create an initial random population.
2. Evaluate fitness.
3. Select parents (mating pool).
4. Apply crossover with probability Pc (typically 0.6 to 0.9).
5. Apply mutation with probability Pm (typically 0.001 to 0.05).
6. Replace the old population (generational or steady-state, with elitism).
7. Stop when a termination condition is met (maximum generations, fitness reached, no improvement); otherwise go to step 2.

### Selection methods

| Method | Idea |
|---|---|
| Roulette wheel | probability proportional to fitness $p_i = f_i/\Sigma f$ |
| Rank | probability based on rank, not raw fitness (avoids domination) |
| Tournament | pick k at random, the best one wins |
| Boltzmann | selection pressure controlled by a temperature |
| Steady-state | only a few individuals replaced each generation |

| Method | Idea |
|---|---|
| Elitism | best individuals always kept |

**Crossover types:** single-point, two-point, multi-point, uniform (each gene from either parent with probability 0.5), and for permutations: PMX (partially mapped), order crossover, cycle crossover.

**Mutation types:** bit flip, swap (exchange two genes), inversion, scramble, and creep or random-value mutation for real numbers.

**GA vs traditional optimization**

| Point | Traditional | GA |
|---|---|---|
| Works on | one point | a population of points |
| Needs | derivatives or problem details | only fitness values |
| Rules | deterministic | probabilistic |
| Parameters | used directly | encoded |
| Local optima | often trapped | more likely to escape |

**Applications:** function optimization, travelling salesman and scheduling, timetable generation, neural network weight and structure optimization, feature selection, robot path planning, engineering design.

## 🧮 Solved examples

**Example 1 (maximize $f(x) = x^2$ for x in 0 to 31, one generation).**

| No. | String | x | f(x) | $p_i = f/\Sigma f$ | Expected count |
|---|---|---|---|---|---|
| 1 | 01101 | 13 | 169 | 0.14 | 0.58 |
| 2 | 11000 | 24 | 576 | 0.49 | 1.97 |
| 3 | 01000 | 8 | 64 | 0.05 | 0.22 |
| 4 | 10011 | 19 | 361 | 0.31 | 1.23 |
| | | | Σ = 1170, avg 292.5 | | |

- Mating pool after selection (rounding expected counts): 01101, 11000, 11000, 10011.
- Crossover 0110|1 with 1100|0 → 01100 (12) and 11001 (25).
- Crossover 11|000 with 10|011 → 11011 (27) and 10000 (16).
- New fitness: 144, 625, 729, 256; sum = 1754, average **438.5**, up from 292.5; maximum 729, up from 576.

**Example 2 (two-point crossover).** Parents 1101|100|1 and 0110|011|0 (cut after bits 4 and 7): children **1101 011 1** = 11010111 and **0110 100 0** = 01101000.

**Example 3 (uniform crossover).** Mask 10110: take a gene from parent 1 where the mask is 1 and parent 2 where it is 0. P1 = 11111, P2 = 00000 → child 10110.

**Example 4 (tournament).** Fitness of A 15, B 40, C 25; tournament of size 2 between A and C picks C.

### Formulas to remember

- $p_i = f_i/\Sigma f$; expected copies = $f_i/\bar{f}$ ($\bar{f}$ = average fitness).
- Schema theorem: $m(H, t+1) \geq m(H, t) \cdot f(H)/\bar{f} \cdot [1 - Pc\ \delta(H)/(l-1) - o(H)\ Pm]$, where $\delta$ is the defining length and o the order.

### ⭐ Important questions

1. Explain the basic concepts of GA and compare it with traditional optimization methods.
2. Explain encoding methods and selection methods (roulette wheel, rank, tournament, elitism).
3. Explain crossover and mutation operators with examples.
4. Explain the generational cycle of GA with a flowchart; solve one generation for a given function.
5. Discuss applications of GA (TSP, scheduling, NN training) and the schema theorem.

## Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Soft vs hard computing; biological vs artificial neuron table.
- ☐ Activation functions (binary/bipolar step and sigmoid with derivatives).
- ☐ Architectures; M-P neuron for AND/OR; linear separability; XOR.
- ☐ Hebb AND table (w1 = 2, w2 = 2, b = −2); perceptron and delta rules.
- ☐ Associative memory: auto vs hetero; Hopfield.
- ☐ Backprop steps; $\delta$ formulas; momentum; factors affecting training.
- ☐ Crisp vs fuzzy; union max, intersection min, complement $1 - \mu$; laws that fail.
- ☐ $\alpha$-cut; Cartesian product; max-min composition example.
- ☐ Triangular, trapezoidal, Gaussian MFs; fuzzification; hedges.
- ☐ Mamdani steps; Sugeno difference; centroid 2.73 example; mean of maximum.
- ☐ FLC block diagram and an application.
- ☐ GA cycle; roulette wheel table for $x^2$; crossover and mutation types; GA vs traditional.

## Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

| # | Title | Author(s) | Publisher | Where to get it legally |
|---|---|---|---|---|
| 1 | Neural Networks, Fuzzy Logic and Genetic Algorithms: Synthesis and Applications | S. Rajasekaran, G. A. Vijayalakshmi Pai | PHI | WorldCat |
| 2 | Artificial Intelligence and Intelligent Systems | N. P. Padhy | Oxford University Press | WorldCat |
| 3 | Neural Networks: A Comprehensive Foundation | Simon Haykin | PHI | WorldCat |
| 4 | Soft Computing: Fundamentals, Techniques and Applications | Saroj Kaushik, Sunita Tiwari | McGraw Hill Education | WorldCat |
| 5 | Fuzzy Logic with Engineering Applications | Timothy J. Ross | Wiley India | WorldCat |
| 6 | Neural Networks: A Classroom Approach | Satish Kumar | Tata McGraw-Hill | WorldCat |

**Syllabus source:** AKTU B.Tech 3rd year syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER _sheerat_